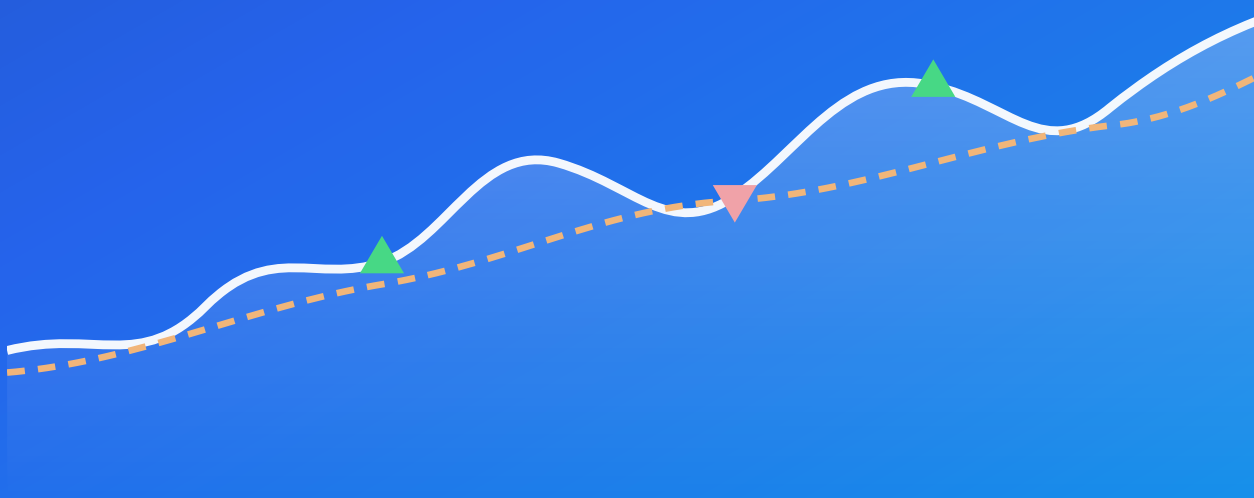
MANTHAN 2026 · TEAM QUBIX

The Qubix Trading Bot

A beginner's guide: what we built, how it works, why it makes the choices it makes, and how to run it, explained in plain words with examples.

No finance or coding background needed



JNNCE Shivamogga · MANTHAN 2026 hackathon

Project: autonomous trading bot for a simulated stock exchange

Prepared for the team · October 2026

What is inside

Read it in order the first time. Every part builds on the one before. Words in **bold** are explained where they first appear, and again in the glossary at the end.

PART 1
The big picture
What the project is, what the hackathon asks for, and stock-market basics.

PART 2
How the bot talks to the market
Prices and news coming in, orders going out, and the timeline of one trading day.

PART 3
The trading strategy
The moving average, the ±1.5% band, how much to buy, and the news check.

PART 4
Safety and reliability
The risk gate, unclear answers, the exchange as the source of truth.

PART 5
Fees and taxes
What every trade costs and how Indian capital-gains tax is estimated.

PART 6
The dashboard
Every page of the live website and how to read each chart.

PART 7
How we chose the strategy
Testing on 500 recorded days, the results, what failed, and honest limits.

PART 8
Running it yourself
Setup, commands, the folder map and the test suite.

PART 9
Questions and glossary
The questions judges and teammates ask, and every term in one place.

THE WHOLE PROJECT IN ONE PARAGRAPH

The organisers run a **pretend stock market** with 10 made-up companies. Every few seconds a new "day" happens: the market announces each company's price and some news headlines. We start with **₹10,00,000 (10 lakh rupees)** of pretend money. Our program, the **bot**, listens to every price and headline and decides on its own, with no human clicking anything, whether to buy or sell. It follows one simple, tested rule: **buy a stock when its price jumps clearly above its recent average, sell it when the price drops clearly below**, and skip a buy when the recent news about that company is bad. A website (the **dashboard**) shows everything it does and explains every decision.

TESTED ON	RESULT IN THAT TEST	WORST FALL FROM A PEAK
500 days	**+34.4%**	**10.0%**
10 stocks · 8,126 headlines	₹10L grew to ₹13.44L	the market fell 13% in one quarter

PART 1

The big picture

Before any code: what a stock market is, what the organisers built for us, and what they want our bot to do.

1.1 Stock market basics in five minutes

A **company** can be split into millions of small pieces called **shares**. If you own a share you own a tiny part of the company. People buy and sell shares all day at a place called a **stock exchange**. The **price** of a share changes all the time: when many people want to buy, the price goes up; when many want to sell, it goes down.

WORD	WHAT IT MEANS	TINY EXAMPLE
Ticker	A short code name for a company's shares.	`HAVN`, `PLNX`
Buy / Sell	Buying gives you shares in exchange for cash; selling turns shares back into cash.	Buy 100 shares at ₹50 = pay ₹5,000
Closing price	The price at the end of a trading day. Our bot works with one price per day: the close.	HAVN closed at ₹135.20
Profit / loss	Sell price minus buy price, times the number of shares, minus costs.	(₹55 − ₹50) × 100 = ₹500 profit
Portfolio	Everything you own right now: cash plus all your shares.	₹6L cash + 3 stocks worth ₹4L
Position	How many shares of one company you hold.	"We have a 700-share position in SONA"
Total wealth	Cash + the current value of all shares. This is the score that matters.	₹10L at the start
Fees / charges	The exchange charges a small fee for every order.	₹20 per order
Tax	In India you pay tax on profit from selling shares.	15% of a short-term profit

REMEMBER

You only **make** or **lose** money for real when you sell. Until then the profit or loss is "on paper" and is called **unrealised**. After the sale it becomes **realised**.

1.2 What the organisers built for us

MANTHAN 2026 is a hackathon. The organisers built a **simulated stock exchange**: it behaves like a real one, but the companies are made up and the money is pretend. Each team gets the same market and the same starting money, and the team whose bot trades best wins.

10 made-up companies

AERX, BLUX, DSNY, HAVN, HMDP, MOTR, ORCX, PLNX, ROUT and SONA. The prices come from real historical market data with the names hidden.

Fast-forward time

One market "day" passes every few seconds (4 seconds in the live run, 2 to 10 seconds in practice runs), so years of trading happen in about an hour.

₹10,00,000 to start

Every team starts with the same pretend cash and no shares.

News headlines

Real news articles, attached to a company, arrive a fraction of a second after each day's prices.

The organisers also run a **portal** (a website) where each team can see its official balance, orders and score. Our own dashboard shows the same numbers, read directly from the exchange.

1.3 What our bot must do

1. **Listen** to the live stream of prices and news for all 10 companies.
2. **Answer the roll call.** Before the real run, the exchange checks every bot is alive by sending numbered "are you there?" messages. The bot must reply with the same number within 15 seconds.
3. **Decide** by itself, every day, whether to buy, sell or do nothing for each company.
4. **Place orders** quickly. A decision is only useful within about 3 seconds of the prices arriving.
5. **Respect the rules:** no spending more cash than we have, no selling shares we do not own, at most 120 requests a minute, and only trade while the simulation is running.
6. **Explain itself.** The judges want to know *why* every trade happened.

1.4 How the judges score us: the five pillars

#	PILLAR	WHAT IT MEANS FOR US
1	**Technical implementation and reliability**	The bot must never crash, double-buy or lose track of what it owns, even if the network drops.
2	**Strategy innovation and analytical depth**	A sensible trading rule, backed by evidence and not by guesswork.
3	**Execution speed and capital efficiency**	React fast, and keep the money working instead of sitting idle.
4	**Market frictions and tax optimisation**	Account for fees and Indian capital-gains tax, and reduce them where it is legal and sensible.
5	**Coordination and explainability**	A clear team split, and an explanation for every decision.

The judges asked for **steady, moderate gains with protection when the market falls**, not wild bets. That is why our bot is deliberately simple, cautious and well tested rather than clever and risky.

PART 2

How the bot talks to the market

Information flows in two directions: prices and news come **in** through one channel, and orders go **out** through another. Here is the whole system on one page.

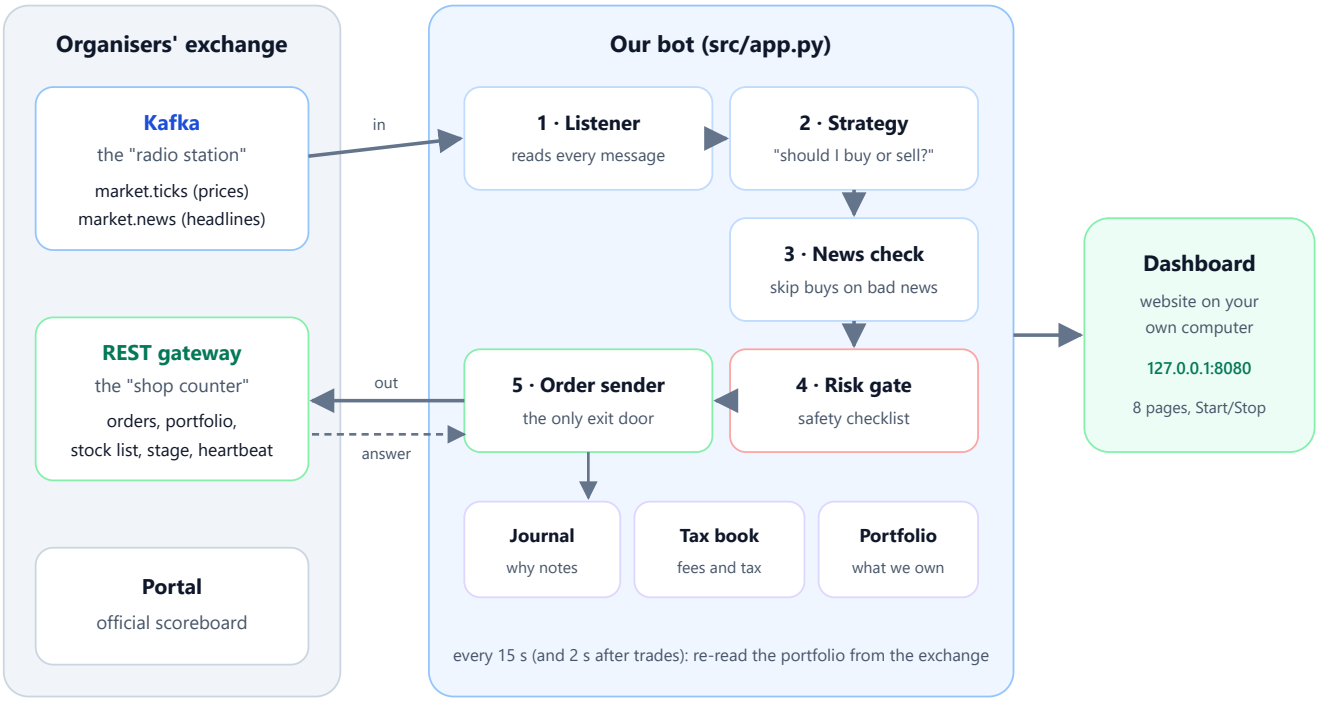


Figure 1. Prices and news arrive from Kafka (left, top). The bot decides, checks the news and the safety list, then sends the order through the gateway (left, middle). The exchange answers, the bot updates its notebooks, and the dashboard (right) shows everything.

2.1 Kafka: the radio station for prices and news

Kafka is a messaging system. Think of it as a radio station with channels, which Kafka calls **topics**. Our bot "tunes in" to two of them:

- `market.ticks` : one message per company per day with that day's prices (open, high, low, close and volume). These messages are called **ticks**. There are also special **pre-flight ticks**, the roll-call messages described below.
- `market.news` : news headlines, each tagged with the company it is about.

Each team listens under its own name (a **consumer group** called `group-mn2631` for us), always starts from the **latest** message (we never replay old days), and if the connection drops it reconnects by itself, waiting 1, 2, 4, 8 and then 15 seconds between attempts. Nothing the bot knows is lost during a reconnect.

2.2 The REST gateway: the shop counter

To *do* something (buy, sell, ask "what do I own?") the bot sends a web request to the organisers' **REST gateway**, like filling in a form at a counter and getting a receipt back.

REQUEST	WHAT IT IS FOR
`POST /api/v1/orders`	Place an order: which stock, BUY or SELL, how many shares (a whole number from 1 to 1,000,000), and the day.
`GET /api/v1/portfolio`	"What do I own?" Cash, shares, profit, charges and the number of orders, exactly as the portal shows them.
`GET /api/v1/stocks`	The list of companies we are allowed to trade.
`GET /api/v1/stocks/<ticker>/history`	Past prices, so the bot can calculate averages from the very first day (the **warm start**).
`GET /api/v1/stage`	Is the simulation in pre-flight, running, paused or finished? The bot only trades while it is running.
`POST /api/v1/bot/heartbeat`	The roll-call reply (see 2.4).

Every request carries our team's secret **access token**, like a password. It lives only in a private file called `.env` on our own computer and is never shown on the dashboard, printed or uploaded.

SPEED LIMIT

The gateway allows **120 requests per minute**. Going over it earns a 30-second "penalty box" during which every request is refused. Our bot limits itself to **100 per minute** to stay safely under, and roll-call replies are allowed to skip the queue so they are never late.

2.3 The timeline of one trading day

Everything in a day happens in a few seconds. Here is what happens on our side, step by step:

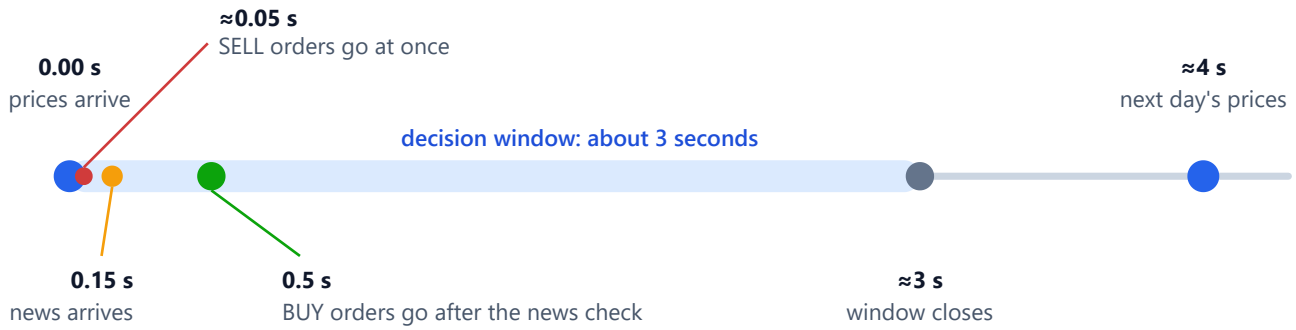


Figure 2. One simulated day. Orders are filled at that day's **closing price**, so a buy sent at 0.5 s costs exactly the same as one sent at 0.01 s. Waiting half a second for the news is free.

EXAMPLE

Day 2025-02-14 begins. At 0.00 s the bot receives MOTR's close of ₹48.37. Its rule says "buy". At 0.15 s the day's headlines arrive; none of them are bad for MOTR. At 0.5 s the bot sends "BUY 1,550 MOTR". The exchange answers **FILLED at ₹48.37** 563 milliseconds after the prices arrived, well inside the window.

2.4 The roll call: pre-flight heartbeat

Before the real run starts there is a **pre-flight** phase. The exchange sends numbered roll-call messages (`seq` 1, 2, 3...) and every bot must reply "present, number 208" within 15 seconds. A bot that does not answer is treated as not ready. Our bot answers automatically, at most once every 2 seconds (pre-flight can send about 20 messages a second, and answering each one would waste our request budget), and retries after 1 second if a reply fails. No trading happens during pre-flight.

IN SIMPLE WORDS

Kafka = the bot's ears. The gateway = the bot's hands. The heartbeat = raising your hand at roll call. The token = the team's ID card.

PART 3

The trading strategy

The **strategy** is the rule that decides when to buy and sell. Ours is called `sma_news`. It has four ingredients: an average, a band, a fixed amount per buy, and a news check.

3.1 Ingredient 1: the moving average

Daily prices jump around a lot. To see the real direction we use a **moving average**: the average of the last 8 closing prices. Each new day, the oldest price drops out and the newest comes in, so the average "moves" along with time. Its short name is **SMA8** (Simple Moving Average of 8 days).

WORKED EXAMPLE

The last 8 closes of a stock were ₹100, ₹102, ₹101, ₹99, ₹100, ₹103, ₹104 and ₹103.
Add them: 100 + 102 + 101 + 99 + 100 + 103 + 104 + 103 = **₹812**. Divide by 8: 812 ÷ 8 = **₹101.50**. That is the SMA8.

Detail: the bot uses the 8 closes *before* today to calculate the average, then compares today's close with it. Today's close only joins the average from tomorrow on.

3.2 Ingredient 2: the ±1.5% band

Around the average we draw two lines, 1.5% above and 1.5% below. Together they form the **band**.

- **Buy line** = average × 1.015. If today's close finishes **above** it, the price has broken out upward: BUY.
- **Sell line** = average × 0.985. If today's close finishes **below** it and we own the stock, the price has broken down: SELL all of it.
- Anywhere inside the band: do nothing. Small wiggles are ignored on purpose.

CONTINUING THE EXAMPLE

Average ₹101.50 → buy line = 101.50 × 1.015 = **₹103.02**; sell line = 101.50 × 0.985 = **₹99.98**.
Today's close ₹105.00 is above ₹103.02, so the rule says **BUY**. Had it closed at ₹99.50 (below ₹99.98) while we held shares, the rule would say **SELL**. At ₹101 or ₹103 it would do nothing.

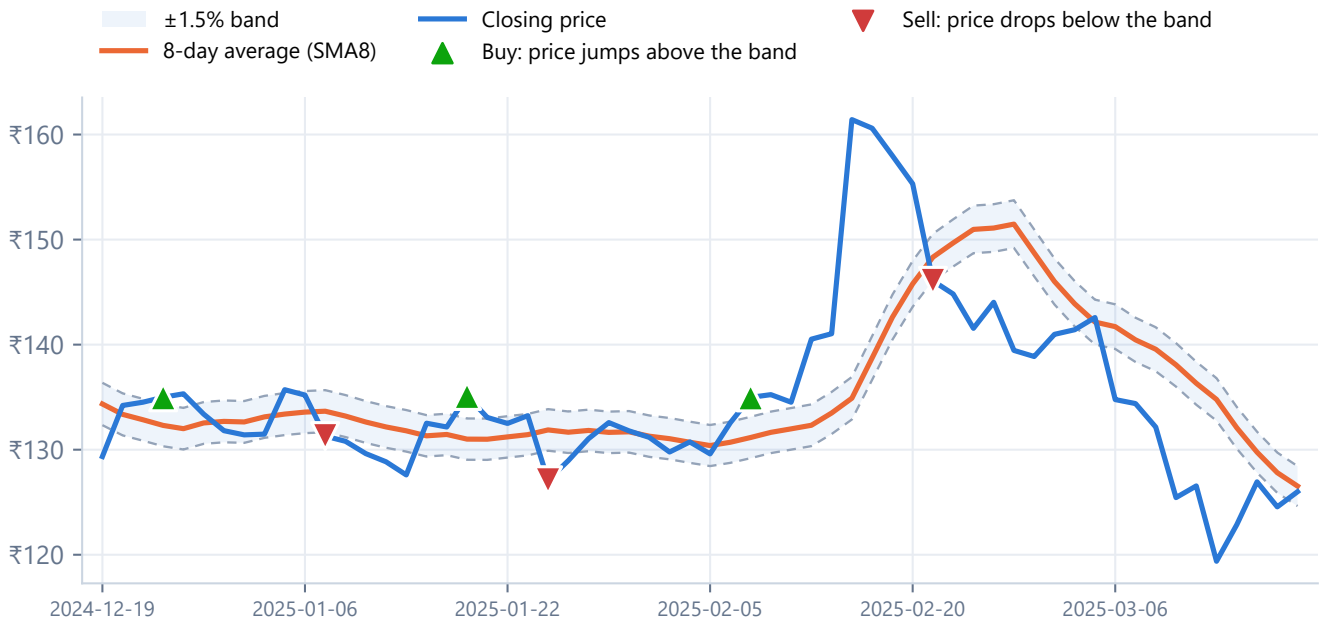


Figure 3. The rule on a real stock from the recording (HAVN, December 2024 to March 2025). Blue is the closing price, orange the 8-day average and the shaded strip the ±1.5% band. Green ▲ marks where the close jumped above the band (buy) and red ▼ where it fell below (sell). Some trades win and some lose: the rule makes money because winners tend to run further than losers.

WHY THIS RULE?

In this market a stock that closes clearly above its recent average often keeps rising for a while, and a stock that falls clearly below often keeps falling. The rule rides the rises and steps aside during the falls. We tested windows from 5 to 12 days and bands from 1% to 2.5%: **every combination made money** over the 500 days. We picked 8 days and 1.5% because it sits in the middle of that safe area, not because it was the single luckiest setting.

3.3 Ingredient 3: how much to buy

- Every buy is worth about **₹75,000**. Number of shares = ₹75,000 ÷ price, rounded **down** to a whole number (at least 1).
- We never hold more than **₹1,50,000** of any one company. So a stock can be bought at most twice before a sale.
- A sale always sells **the whole position** at once.

EXAMPLE

Price ₹105 → 75,000 ÷ 105 = 714.28 → **714 shares**, costing 714 × ₹105 = **₹74,970** plus a ₹20 fee. A second buy is allowed only if the total stays under ₹1,50,000.

Why ₹75,000? Bigger buys mean bigger profits *and* bigger losses. We tested several sizes: the original bot used ₹50,000 (+17.9% over 500 days, worst fall 7%); ₹75,000 gave +28.8% with a worst fall of about 10%. Doubling the size again gave only a little more return for noticeably more risk, so we stopped at 1.5×.

3.4 Ingredient 4: the news check (the "veto")

Headlines arrive 0.15 s after the prices. When the rule wants to **buy**, the bot waits **0.5 seconds** so that day's headlines are in, then asks: *has the news about this company been bad lately?*

How a headline gets a score

The bot reads each headline (and its short description) and looks for warning words and good words:

TYPE OF WORD	POINTS	EXAMPLES
Severe bad news	−2 each	fraud, bankruptcy, probe, lawsuit, recall, breach, hack, crash, plunge, ban
Mild bad news	−1 each	downgrade, miss, loss, weak, cuts, layoffs, slump, decline, warns, delay, outage
Good news	+1 each	record, surge, upgrade, beat, profit, wins, expands, strong, growth, approved
"Flip" words	flip the sign	no, not, never, without, denies, avoids, fails, despite (within 3 words before)

HEADLINE A

"Regulator opens **probe** after data **breach**"
probe −2, breach −2 → score **−4**

HEADLINE B

"Company **denies fraud** claims"
fraud is −2, but "denies" just before it flips it → **+2**

One headline can score from −4 to +4. The bot adds up the scores of all headlines about that company from the last **3 days**. If the total is **−2 or worse**, the buy is **skipped**. Sells never wait and are never skipped: getting out quickly matters more.

WHY ONLY A VETO, AND NOT "BUY ON GOOD NEWS"?

We checked 8,126 headlines. On their own, they predict almost nothing about tomorrow's price (the correlation was about −0.01 to −0.03, basically zero). Most are ordinary news stories with the company name hidden. But one pattern was real: **a price breakout that happens on bad news usually fails**. So the news is used only to skip risky buys. Skipping 52 such buys raised the 500-day result from +28.8% to +34.4%. To make sure this was not luck, we re-ran the test 40 times with the headlines moved to the wrong company or the wrong date ("placebo" runs); the real version beat all 40.

3.5 Extra: year-end tax-loss harvesting

In India the tax year runs from 1 April to 31 March. In the **last 10 days before 31 March**, if we have made taxable profit that year, the bot looks for stocks we hold at a loss. Selling such a stock turns the paper loss into a real one, which reduces this year's taxable profit. It does this only when the loss is at least ₹1,000 and the tax saved is bigger than the ₹55.34 cost of selling and buying back. It is a small, legal optimisation (about +0.6 percentage points in the test), and we say so honestly.

3.6 The whole rule, in one box

SMA_NEWS, COMPLETELY

1. Every day, for every stock: work out the average of the previous 8 closes.
2. Close more than 1.5% above the average, and below the ₹1.5L limit? Prepare a ₹75,000 **buy**.
3. Wait 0.5 s. If that company's last-3-days news score is −2 or worse, **skip** the buy.
4. Close more than 1.5% below the average, and we own it? **Sell everything**, immediately.
5. In the last 10 days of the tax year, sell losing positions when the tax saving beats the cost.
6. Every order passes the safety checklist (Part 4) before it is sent.

PART 4

Safety and reliability

A trading bot that makes one mistake, like buying twice or selling shares it doesn't have, can ruin a good strategy. Most of our code is about **not** making mistakes.

4.1 The risk gate: a checklist before every order

No order can leave the bot without passing through one function, the **risk gate** (`src/risk/manager.py`). It works like a pilot's pre-flight checklist. If any answer is "no", the order is not sent and the reason is written in the journal.

#	CHECK	WHY
1	Is the simulation running?	Orders during pre-flight or pause are pointless or rejected.
2	Is it a BUY or a SELL?	We never "short sell" (sell shares we don't own).
3	Is the quantity a whole number from 1 up?	Half-shares or zero shares are invalid.
4	Is this stock on the tradable list?	Unknown tickers are refused.
5	Is the price valid and positive?	A broken price would give a nonsense order.
6	Are the prices fresh?	Never trade on stale data.
7	Do we know our portfolio?	We must know what we own before changing it.
8	Is an earlier order for this stock still unclear?	Wait until we know (see 4.2) so we never double-buy.
9	Did we already send this exact order today?	Stops accidental duplicates.
10	Selling: do we own at least that many shares?	You cannot sell what you do not have.
11	Buying: will we stay under ₹1,50,000 in this stock?	No single company can hurt us too much.
12	Buying: do we have enough cash, fees included?	The exchange would reject it anyway; we catch it first.

Because the gate checks cash and limits first, the exchange had to reject **0** of our orders in the 500-day test and 0 in the live practice audit of 1,153 orders.

4.2 Three possible answers: FILLED, REJECTED, UNCLEAR

FILLED

The exchange did it. The bot updates its portfolio, tax book and journal.

REJECTED

The exchange said no and gave a reason code. Nothing changed; the reason is logged.

UNCERTAIN

No clear answer (the network timed out). The order may or may not have happened.

"UNKNOWN IS NOT NO"

Many bots treat a timeout as "it failed" and try again, and end up buying twice. Ours freezes that stock, asks the exchange "what do I actually own?", and only then trades it again.

4.3 The exchange is the source of truth

The bot keeps its own notes, but the exchange's records always win. It re-reads the portfolio from the exchange at start-up, **every 15 seconds**, **2 seconds after each burst of trades**, after any unclear answer, and after any "not enough cash" rejection. If a re-read fails, it waits 5 seconds before trying again so it does not flood the gateway. That is also why our dashboard shows the same numbers as the organisers' portal.

Account reset warning. The exchange's order count for our account normally only goes up. If it ever goes *down*, someone reset the account on the exchange side. The bot then prints a warning and the dashboard shows an amber banner, so nobody is confused by numbers that suddenly don't match.

4.4 Other protections

- **Reconnects by itself** if Kafka drops, without forgetting anything.
- **Stop is not unplug.** The dashboard's Stop button pauses new orders only. The bot keeps listening and keeps answering the roll call, so it is ready to restart instantly.
- **Only one bot at a time.** A second copy started by mistake refuses to run, so two bots can never trade the same account.
- **The dashboard cannot break trading.** If the website code fails, the bot carries on.
- **Local only.** The dashboard is reachable only from your own computer (address 127.0.0.1) and never sees the token.
- **Logging never crashes.** Headlines with odd characters are printed safely instead of stopping the bot.

PART 5

Fees and taxes

Real trading costs money. Ignoring costs is the most common way a strategy that looks good on paper loses money in practice. Our bot counts every rupee.

5.1 Fees on every order

CHARGE	AMOUNT	WHEN
Brokerage	₹20	on every order, buy or sell
DP charge (depository participant)	₹15.34	on every sell
A full round trip (one buy + one sell)	₹55.34	20 + 20 + 15.34

₹55 sounds small, but a bot that trades 500 times pays thousands. That is why we rejected ideas that traded more often without earning enough extra to pay for it.

5.2 Capital-gains tax (India)

HELD FOR	NAME	TAX RATE
365 days or less	**Short-term capital gain (STCG)**	15% of the profit
More than 365 days	**Long-term capital gain (LTCG)**	10% of the profit, and the first ₹1,00,000 of long-term profit each tax year is tax-free

- **FIFO (first in, first out).** If you bought shares on two different days, a sale is counted as selling the oldest ones first. That decides how long they were held and what they cost.
- **Fees count.** The ₹20 buy fee is added to the cost; the sell fees are taken from what you receive.
- **Losses offset gains** within the same tax year (Sections 70 and 71): short-term losses reduce short-term gains first, then long-term ones; long-term losses can only reduce long-term gains.
- The tax year runs 1 April to 31 March of the simulated calendar.

A COMPLETE TRADE WITH COSTS AND TAX

Buy 714 shares at ₹105	₹74,970.00
+ brokerage	₹20.00
Total cost	**₹74,990.00**
20 days later, sell 714 shares at ₹112	₹79,968.00
− brokerage − DP charge (20 + 15.34)	−₹35.34
Received	**₹79,932.66**
Profit (79,932.66 − 74,990.00)	₹4,942.66
Held 20 days → short-term → tax 15%	₹741.40
Kept after tax	**₹4,201.26**

Our bot calculates tax as an **estimate** for its decisions and the dashboard. The exchange's own tax report is the official one.

PART 6

The dashboard

The dashboard is a website that runs on your own computer while the bot runs. Open `http://127.0.0.1:8080/` in a browser. It has eight pages; switch with the side menu or the keys **1** to **8**. Everything updates by itself every 2 to 3 seconds.

The screenshots below come from the **offline demo**, which replays our recorded days so the pages have data to show. The live dashboard looks exactly the same. The money figures in them are from a demo run partway through, not the 500-day results in Part 7.

6.1 The header (every page)

Top right: **Bot status** (running or not), **Kafka** (connected?), **Heartbeat ack** (the last roll-call number we answered and when), **Simulation** (pre-flight, running, paused, completed) and a badge saying whether this is a practice **Sandbox** or the **Live run**. Bottom left of the menu: whether trading is ON.

6.2 Page 1 · Overview

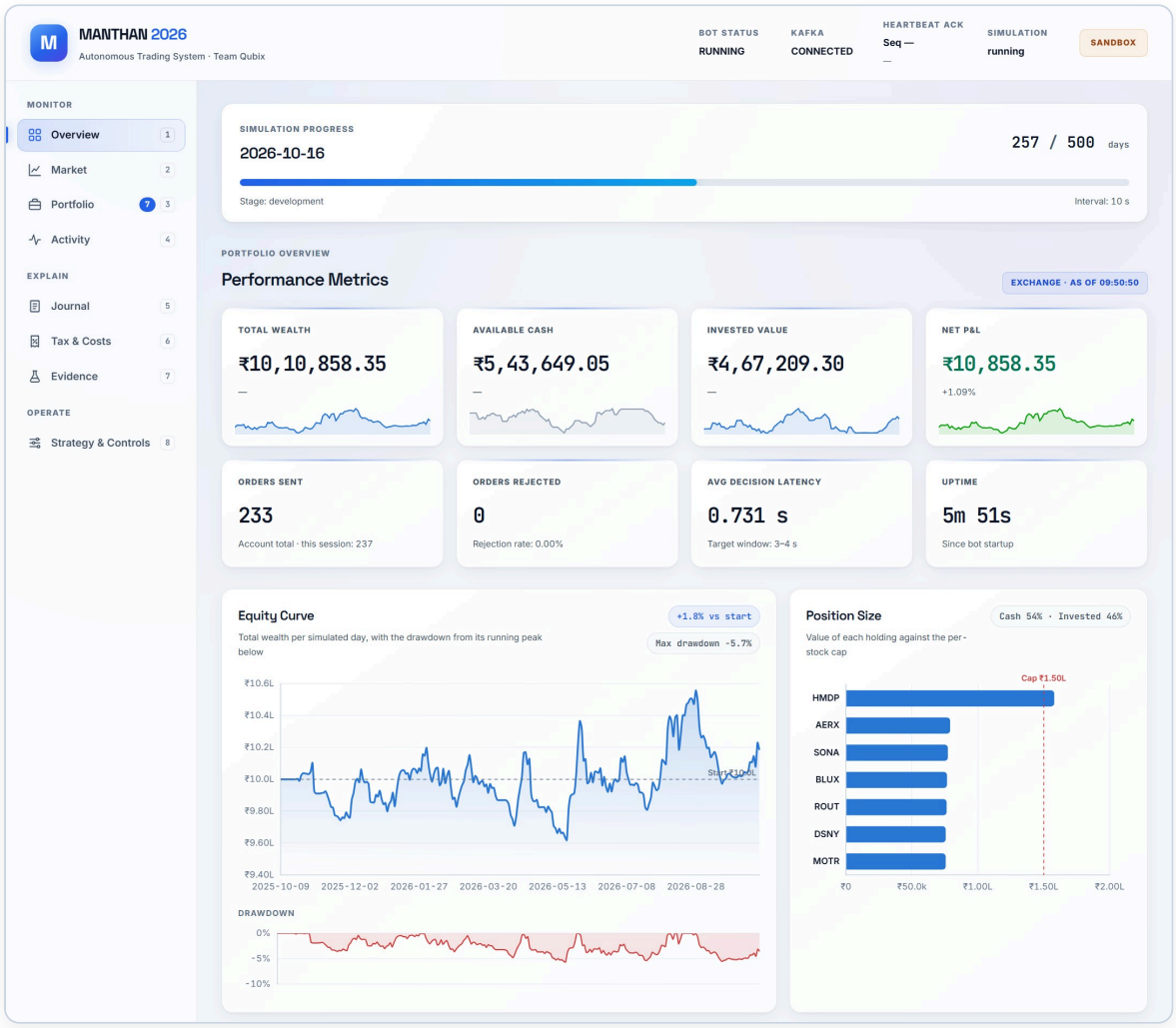


Figure 4. The Overview page: simulation progress, eight headline numbers with small trend lines, and the main charts.

- **Simulation progress**: today's simulated date and how many of the days are done.

- **Eight number cards**: total wealth, cash, invested value, net profit, orders sent, orders rejected, average decision time and uptime. The blue label "EXCHANGE · AS OF 09:44" means the money figures come straight from the exchange and how fresh they are.
- **Equity curve**: our total wealth over time. The dashed line is the ₹10L starting point; above it we are in profit. The red strip below is the **drawdown**: how far we are below our best point so far.
- **Position size**: what each holding is worth compared with the red ₹1.5L limit line.
- **Distance from trigger**: for each stock, how far today's close is from its average. Bars past the green line (+1.5%) are buy signals; bars past the red line (−1.5%) are sell signals if we own them; grey bars are inside the band. A ● after the name means we own it. This chart shows what the bot is likely to do next.

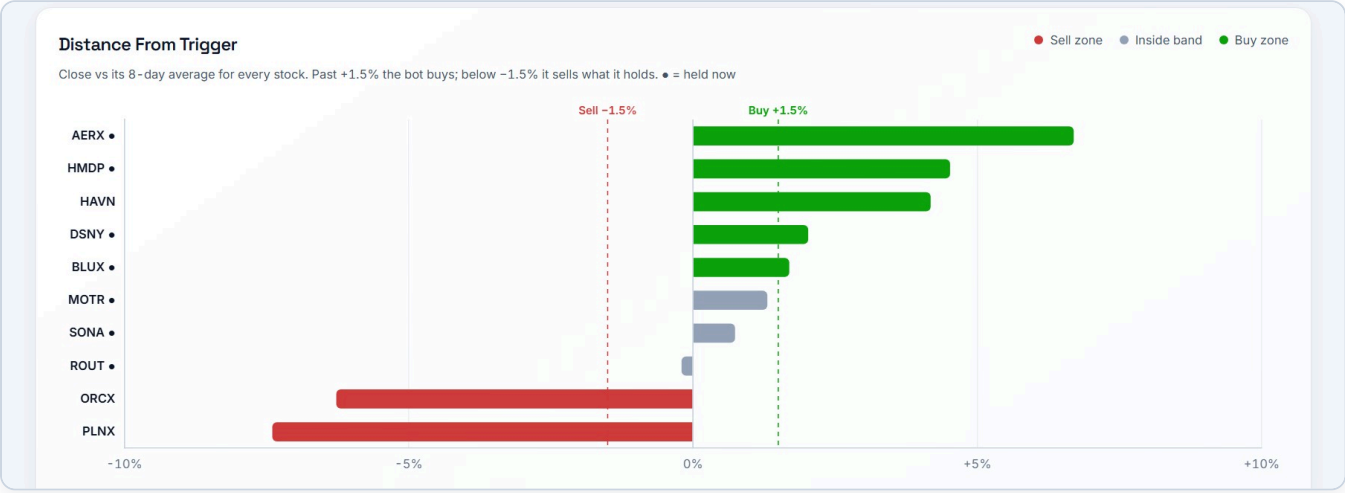


Figure 4b. Distance from trigger. AERX, HMDP, HAVN, DSNY and BLUX are past the buy line; ORCX and PLNX are past the sell line, but we don't hold them (no ●), so there is nothing to sell.

6.3 Page 2 · Market

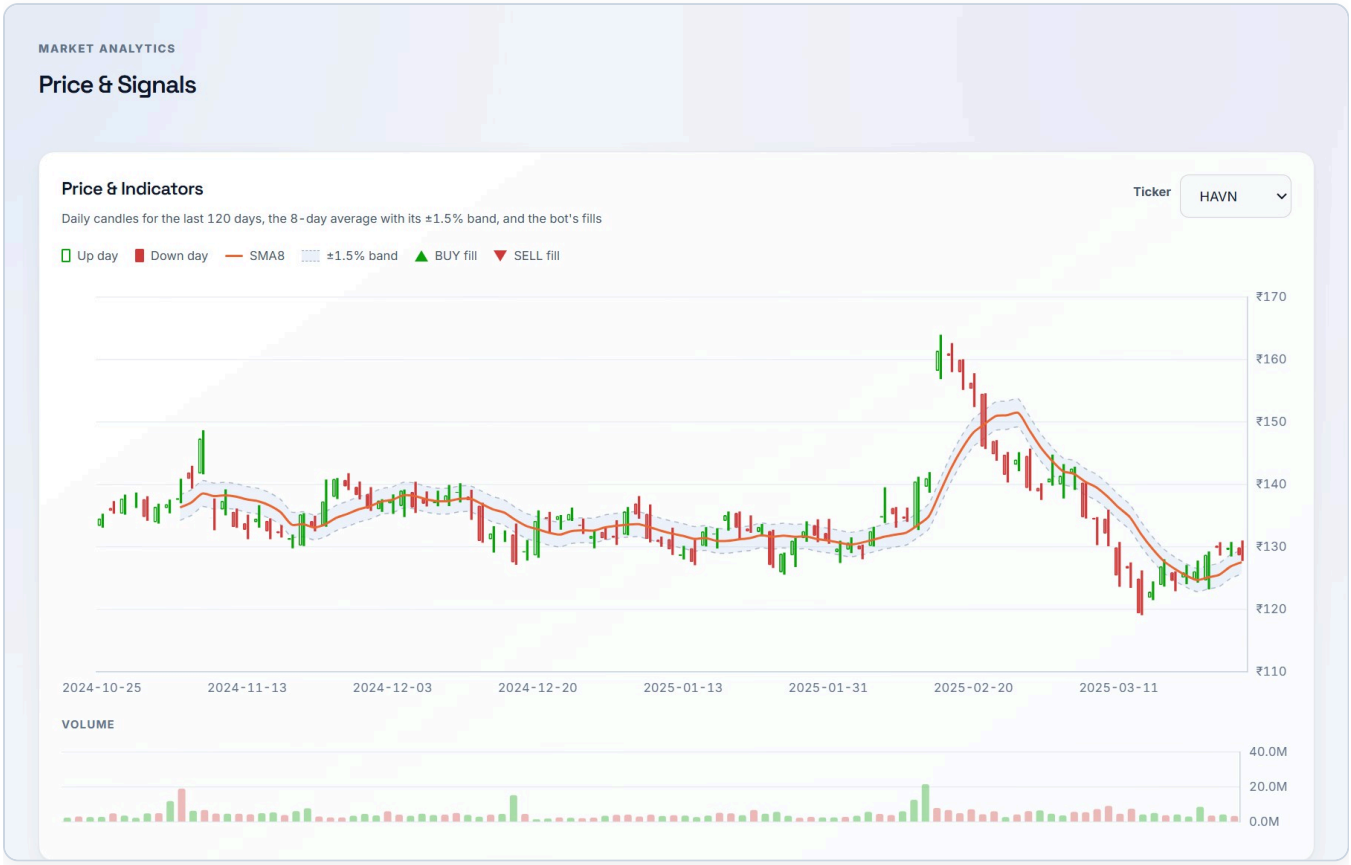


Figure 5. The Market page: daily candles, the 8-day average with its band, and our trades.

Pick a stock from the menu. Each day is drawn as a **candle**: the thin line shows the day's highest and lowest price; the box shows the opening and closing price. A hollow green box is an up day (closed higher than it opened); a filled red box is a down day. The orange line is the 8-day average and the shaded strip is the ±1.5% band. Green ▲ and red ▼ are the bot's actual buys and sells. Below: daily **volume** (how many shares changed hands). Under the chart is the latest news list.

6.4 Page 3 · Portfolio

A table of every stock we hold: quantity, last close, market value, average buy price, unrealised profit, how much of the ₹1.5L limit is used, the date we first bought it and whether it is short- or long-term for tax. Below it, the **Orders** table lists every order this session with the exchange's answer, fees and latency.

6.5 Page 4 · Activity

A live feed of everything the bot does: prices, signals, orders, risk blocks, news, errors and system messages, with filters by type and by stock. Below it are two charts. **Decision latency** shows how many orders took how long (most go out in about 0.1 s or 0.5 to 0.7 s; the window is about 3 s). **Risk blocks** shows why orders were not sent, for example "Skipped: bad news".

6.6 Page 5 · Journal: why each trade happened

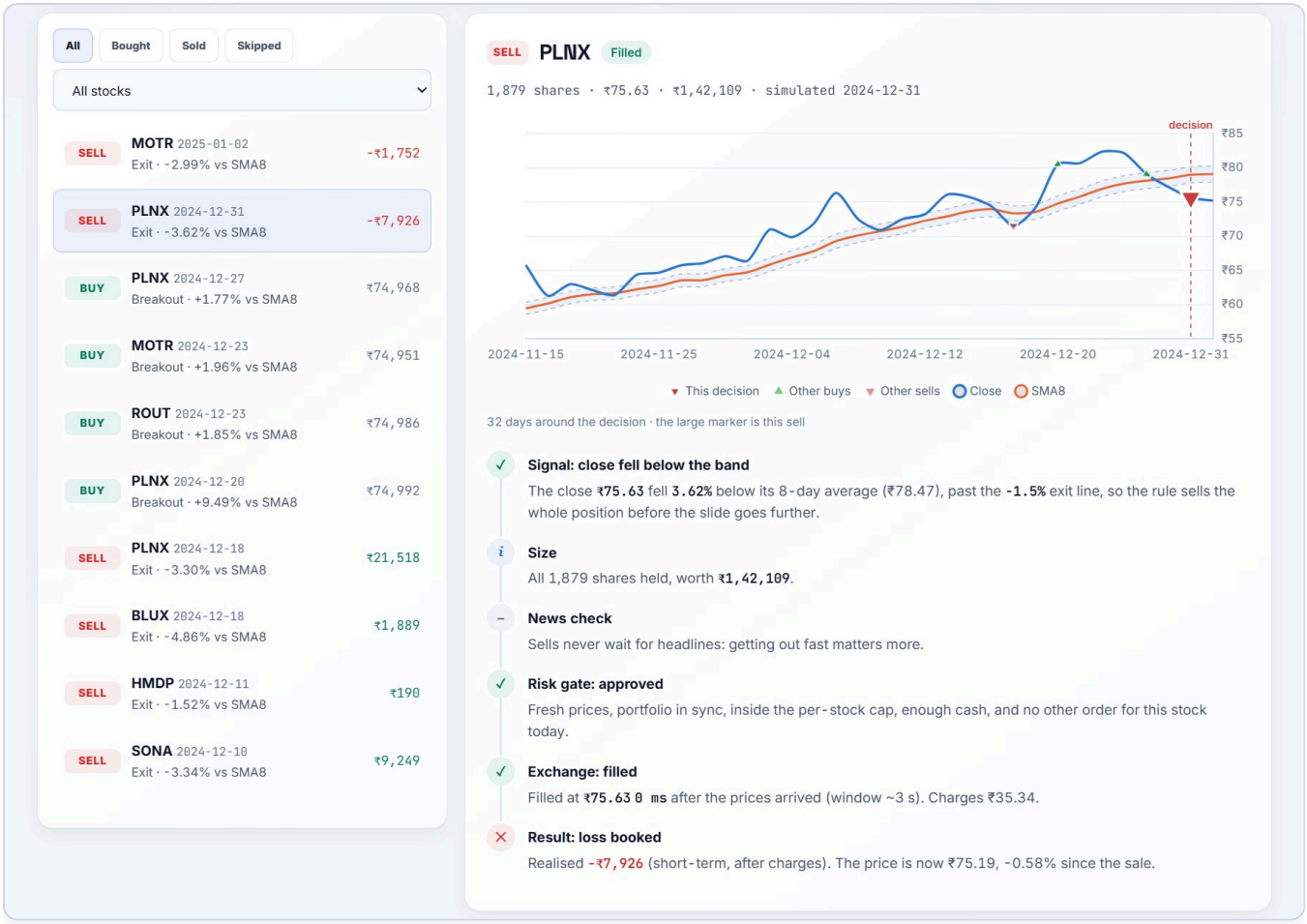


Figure 6. The lower half of the Journal page. Left: every decision, newest first (BUY, SELL or SKIP), with a short reason and its result. Right: the selected decision (a PLNX sale) explained step by step, with the price chart around that day; the large red ▼ is this sale.

This is the page to show judges. At the top: total decisions, closed trades, win rate, realised profit, buys skipped because of news, and average decision time. Then two charts: profit or loss of each closed trade, and how many decisions each rule produced.

Click any decision to see its explanation in plain words:

1. **Signal**: "The close ₹48.37 finished 1.68% above its 8-day average (₹47.57), past the +1.5% buy line."
2. **Size**: "1,550 shares × ₹48.37 = ₹74,974, the fixed amount per buy."
3. **News check**: "Headlines over the last 3 days scored 0, above the −2 veto line." (or why it was skipped)
4. **Risk gate**: approved, or which check blocked it.
5. **Exchange**: "Filled at ₹48.37, 563 ms after the prices arrived. Charges ₹20."
6. **What happened next**: the price since then, the profit on a sale, or, for a skipped buy, whether skipping avoided a loss or missed a gain.

Every decision is also saved to a file (`logs/decisions_*.jsonl`) and can be downloaded as a spreadsheet with the **Download CSV** button.

6.7 Pages 6 to 8

6 · Tax & Costs

Charges paid (from the exchange), estimated short- and long-term tax, tax-loss credits, tax per financial year, and the current harvesting candidates.

7 · Evidence

Our research: the 500-day results, equity curves against buy & hold, quarter-by-quarter results, every strategy we tested, and what we rejected and why.

8 · Strategy & Controls

Start / Stop trading (Stop pauses new orders only), and the strategy selector, which can only be changed while trading is stopped.

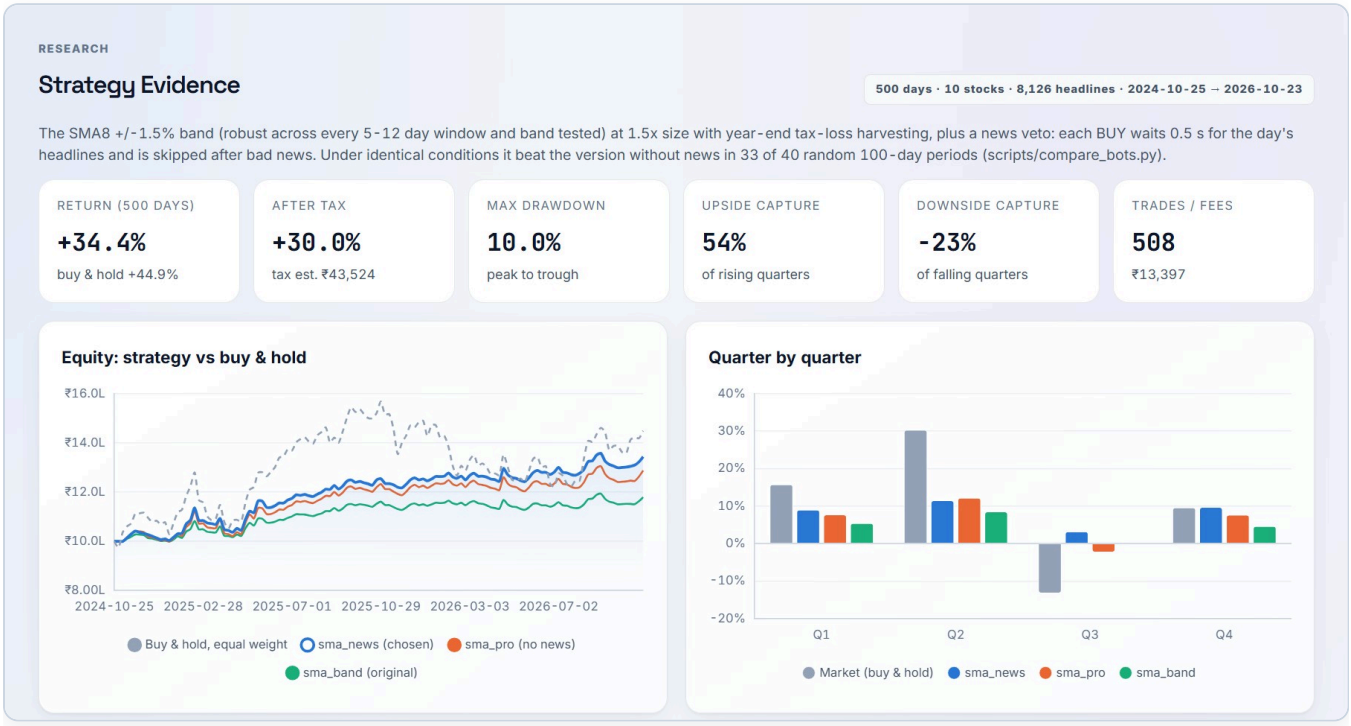


Figure 7. The top of the Evidence page: the chosen strategy's 500-day figures and the two charts from Part 7 of this guide.

PART 7

How we chose the strategy

Anyone can claim a strategy works. We wanted proof, so we tested every idea on real market data before letting it near the live run.

7.1 Backtesting: practising on a recording

During practice runs we **recorded** the full market stream: 500 trading days (25 October 2024 to 23 October 2026) of prices for all 10 stocks, plus 8,126 headlines. Then we **replayed** it through the bot, like practising on last year's exam paper. This is called a **backtest**.

Two things make our backtests trustworthy:

- **It is the real bot.** The replay runs the exact code that trades live, with a pretend clock that copies the live timing (prices at 0 s, news at 0.15 s, about 3.5 s per day) and a pretend exchange that fills at the close and charges the same fees.
- **Hard rules for accepting a change.** A new idea had to beat the current bot in *both halves* of the 500 days, stay positive in all five 100-day stretches, still work without PLNX (one stock that rose 338% and could flatter any strategy), and still work when its settings are nudged a little.

7.2 The results: three versions of the bot

	ORIGINAL `SMA_BAND`	STEP 1 `SMA_PRO`	FINAL `SMA_NEWS`
What it is	₹50k per buy	₹75k per buy + tax harvesting	sma_pro + news veto
Final wealth (from ₹10L)	₹11,79,243	₹12,87,836	**₹13,43,785**
Return over 500 days	**+17.9%**	**+28.8%**	**+34.4%**
After estimated tax	+15.9%	+25.4%	**+30.0%**
Worst fall from a peak (max drawdown)	**7.0%**	10.3%	10.0%
Sharpe ratio (return per unit of ups and downs)	1.06	1.15	**1.39**
Orders / rejected by exchange	552 / 0	548 / 0	508 / 0
Charges paid	₹14,507	₹14,442	**₹13,397**
Win rate (sells in profit)	37%	37%	39%
Buys skipped on bad news	—	—	52

READING THE WIN RATE

Only about 4 in 10 sales are profitable, yet the bot makes money. That is normal for this kind of rule: losing trades are cut quickly and small, while winning trades are held while they keep rising.

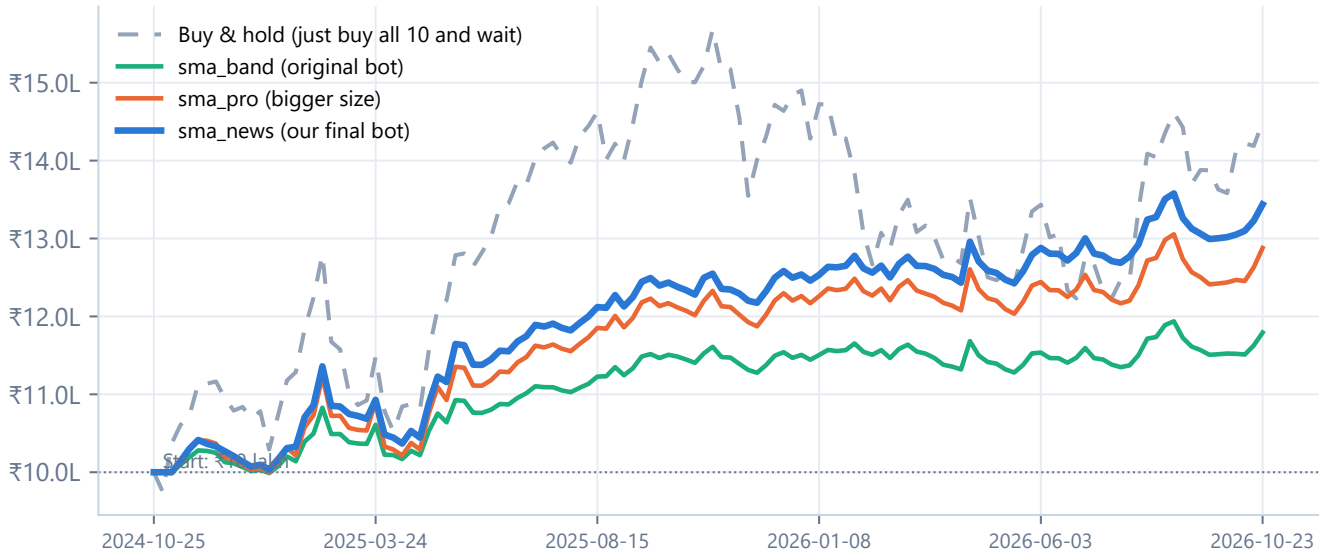


Figure 8. Wealth over the 500 days for the three bots and for simply buying all 10 stocks and waiting (grey dashed). Buy & hold ended higher (+44.9%), but it fell hard in the crash; our bots grew more steadily.

7.3 Protection when the market falls

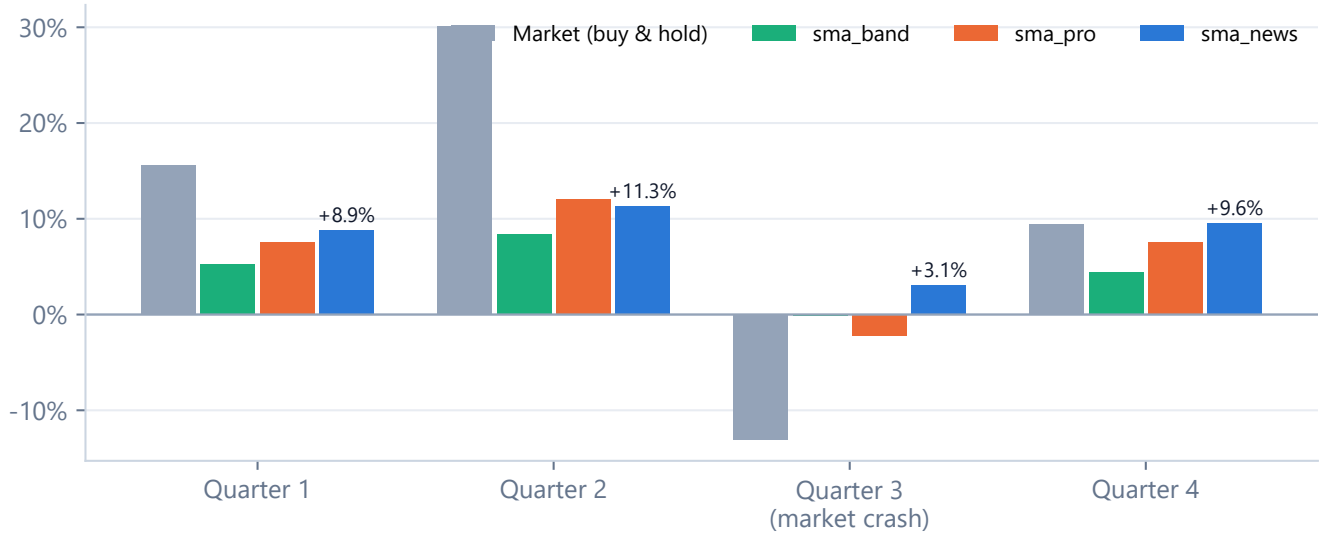


Figure 9. Return in each quarter of the test. In quarter 3 the market fell 13.1%; the final bot made +3.1% in that same quarter. In rising quarters it keeps roughly half of the market's gain.

This is the behaviour the judges asked for: moderate gains when the market rises and protection when it falls. The band rule sells stocks as soon as they drop below their average, so in a crash the bot is mostly holding cash.

7.4 Was it just luck? 40 random periods

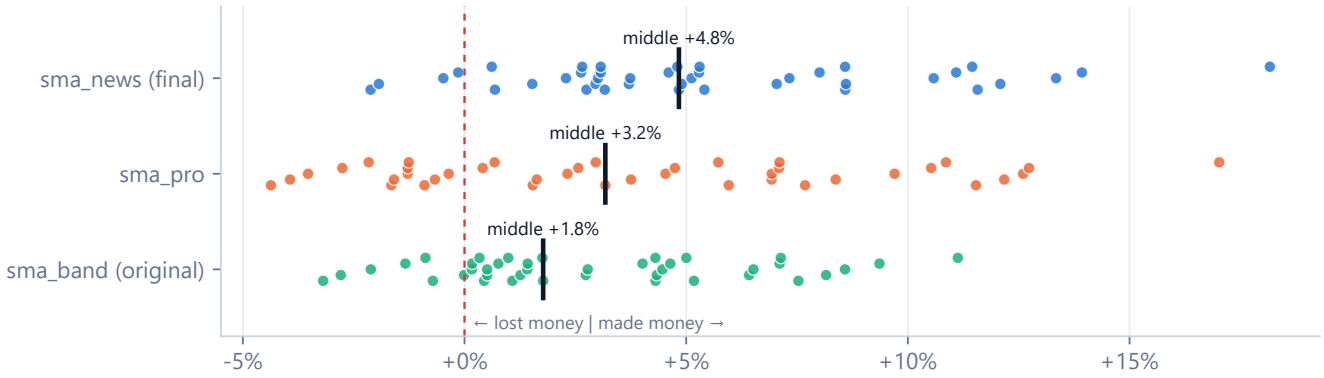


Figure 10. Each dot is the return over one random 100-day stretch (the same 40 stretches for every bot). The black tick marks the middle result.

- The final bot beat `sma_pro` in **33 of 40** periods and the original bot in **40 of 40**.
- It lost money in only **4 of 40** periods (sma_pro: 13, original: 7). Its worst period was −2.1%.
- Without PLNX, the best-performing stock, it still made +25.3%.

7.5 What we tried and threw away

Good research also reports failures. These ideas were tested the same way and rejected:

IDEA	WHY IT WAS REJECTED
Momentum (breakout, relative strength, adaptive bands)	Longer trends in this data reverse; all were near zero or lost money.
Buy-the-dip / laggard ensemble	Best on the days it was designed on, about zero on days it never saw. Classic overfitting.
Market-regime switch	+0.4% with a 10.8% drawdown at every setting.
Trading on news alone	Headlines don't predict next-day prices here.
Stop trading after a 6% account loss	Locked the bot out of the recoveries: +2.5% instead of +28.8%.
14 entry and exit tweaks	Confirmation days, earlier exits, trailing stops, trend and volume filters: none beat the final bot on every check.
13 extra risk limits	Stop-losses, exposure caps, maximum positions, maximum buys per day: none passed every check.
Short selling	No edge found, and not approved in our team rules.

7.6 Honest limits

WHAT THIS EVIDENCE DOES NOT PROVE

- It is **one** 500-day dataset. The live run uses a different period (it starts in 2019 and lasts 1,008 days), so live results will differ. The 40-random-period test is the fairest guide.
- The news gain rests on about 50 skipped trades, so "+6 points" is an estimate, not a promise.
- The practice sandbox replays the same 500 days in a loop, so profits made there are not new evidence.
- The final bot takes more risk than the original (10% vs 7% worst fall) in exchange for about twice the return.
- Our tax figure is an estimate; the exchange's report is official.

PART 8

Running it yourself

Everything below is typed in a terminal (for example the VS Code terminal) inside the project folder.

8.1 One-time setup

1. Install Python 3.11 and open the project folder in VS Code.
2. Install the libraries: `py -m pip install -r requirements.txt`
3. Create a file named `.env` in the project folder with the team's settings:

```
GATEWAY=http://192.168.65.157:3002
BROKER=192.168.65.157:9094
HACKATHON_USERNAME=mn2631
BOT_ACCESS_TOKEN=<the team token: ask the team lead, never share it>
STRATEGY=sma_news
```

4. Be on the hackathon network (the exchange's address only works there).

NEVER

Never paste the token into chat, a screenshot, a commit or this document. `.env` is listed in `.gitignore` so Git will not upload it.

8.2 Commands (PowerShell, e.g. the VS Code terminal)

WHAT YOU WANT	COMMAND
Bot + dashboard, trading ON at once	`$env:START_TRADING = '1'; py -m src.app --live --dashboard`
Bot + dashboard, trading paused until you press Start	`py -m src.app --live --dashboard`
Bot only, no dashboard (trades at once)	`py -m src.app --live`
Offline demo (no exchange, no token needed)	`py -m scripts.demo_dashboard data/stream_20261029_215617.jsonl --strategy sma_news --speed 2 --port 8090 --start`
Run all the tests	`py -m pytest`
Stop the bot	Press **Ctrl + C** in its terminal. It stops cleanly and prints a summary.

Then open `http://127.0.0.1:8080/` (or `:8090` for the demo). Optional settings, typed before the command: `$env:STRATEGY = 'sma_pro'`, `$env:DASHBOARD_PORT = '8081'`. Only one live bot can run at a time.

8.3 What you see in the terminal

```
09:33:31 Sizing: Rs 75,000 per buy, Rs 150,000 max per stock
08:33:31 Dashboard: http://127.0.0.1:8080/  (trading is ON)
08:33:31 Synced holdings: {'ROUT': 2088, 'HAVN': 618, ...} cash: 1326463.21
08:33:31 Connected to Kafka. Listening...
08:33:36 TICK 2025-04-16 BLUX 238.57
08:33:36   BLUX vs SMA8: +2.77% (trigger +/-1.5%)
08:33:36   BUY 314 BLUX waits 0.5s for today's headlines
08:33:36 NEWS BLUX Martin Lee obituary (+1/-0)
08:33:36   news risk score +2 (entries vetoed at <= -2)
08:33:36 ORDER BUY 314 BLUX FILLED  (close above SMA8 by 1.5% (+2.77%))
```

Read it top to bottom: the settings, the dashboard address, what we already own, connection, then each day's prices, the distance from the average, the buy waiting for news, the headline and its score, and the filled order.

8.4 The folder map: what every file does

FILE / FOLDER	WHAT IT DOES
`src/app.py`	The heart: start-up, listening to Kafka, the heartbeat, calling the strategy, sending orders, re-reading the portfolio, reconnecting.
`src/strategy/sma.py`	The trading rule: `sma_band`, `sma_pro` and `sma_news`.
`src/strategy/registry.py`	The list of strategies you can choose by name (default `sma_news`).
`src/strategy/` (others)	Research strategies we tested and rejected, and `backtest.py` for quick replays.
`src/news/filter.py`	Scores headlines and decides the news veto.
`src/risk/manager.py`	The 12-point safety checklist.
`src/execution/client.py`	The only code that talks to the gateway: speed limit, penalty box, FILLED / REJECTED / UNCERTAIN.
`src/portfolio/state.py`	What we own, synced from the exchange; detects account resets.
`src/tax/engine.py`	The tax book: FIFO lots, STCG/LTCG, set-off, harvesting candidates.
`src/journal.py`	Writes the "why" of every decision to `logs/`.
`src/market_data/`	Reads tick messages and records or loads the stream.
`src/dashboard/`	The local web server, its data, and the offline pretend exchange for the demo.
`frontend/`	The dashboard website: `index.html`, `style.css`, `app.js`, `charts.js`, `insights.js`, `pages.js`.
`scripts/`	Tools: record the stream, replay the real bot (`replay_bot.py`), compare bots, build the evidence, run the demo.

`tests/unit/`	189 automatic tests that run without any network.
`docs/`	Research reports and notes (results, architecture, jury defence, demo runbook).
`data/` , `logs/`	Recordings and decision logs. Kept on your computer only, not uploaded.

8.5 Tests: how we know it works

The project has **189 automatic tests**. Each one sets up a small situation with a pretend exchange and checks the bot does the right thing, for example "a timeout must not cause a second buy", "the dashboard shows the exchange's numbers", "a bad-news buy is skipped but a sell is not", "a reset account is flagged". Run them with `py -m pytest`; they take about 15 seconds and need no network. Run them after every change.

8.6 Who owns what

The work is split into four areas so that each person can change their part without breaking the others. The shared interfaces are written down in `CONTRACTS.md`.

Data and strategy

Recording the stream, parsing prices and news, the trading rule and the research.

Execution and risk

The gateway client, the risk gate, portfolio sync, the tax book and reliability.

Dashboard

The website, its pages and charts, and the Start/Stop controls.

QA and demo

Tests, the offline demo, the live-run checklist and presenting to the judges.

PART 9

Questions and glossary

Why not just buy all 10 stocks and wait? That made +44.9%.

It also fell 13% in a single quarter. The judges asked for steady gains with protection on the way down. Buy & hold is a bet that the market rises; on an unknown period we cannot know that. Our bot made +3.1% in the quarter the market fell 13.1%.

Can the bot lose money?

Yes. About 6 in 10 sales lose a little. In 4 of 40 random 100-day periods the final bot ended slightly down (worst −2.1%). The rule wins over time because winners are held and losers are cut quickly.

Why sell the whole position at once?

A close below the band means the rise has probably ended. Selling part would keep money in a falling stock, and every extra order costs fees.

Why does a buy wait 0.5 seconds?

The news arrives 0.15 s after the prices. Waiting 0.5 s lets us read it first, and costs nothing because orders fill at the day's closing price anyway.

What if the network drops in the middle of the run?

Kafka reconnects by itself. An order with no clear answer is marked UNCERTAIN and that stock is frozen until the bot re-reads what it owns, so nothing is bought twice.

What if the dashboard and the portal show different numbers?

The dashboard reads the exchange every 15 seconds and 2 seconds after trades, so they match within seconds. If the account is reset on the exchange side, an amber banner explains it.

Does the bot use AI or machine learning?

No. It uses a simple, transparent rule that we can explain line by line. We tested more complex ideas and they did worse on data they had not seen.

Can we change the strategy during the run?

Yes, from page 8 of the dashboard, but only after pressing Stop. Open positions are then managed by the new rule.

Glossary

Backtest

Testing a strategy by replaying recorded market data.

Bot

A program that trades on its own, without a human clicking.

Band (±1.5%)

The zone between 1.5% above and 1.5% below the average; inside it the bot does nothing.

Brokerage

The ₹20 fee per order.

Candle
A chart shape showing one day's open, high, low and close.

Close (closing price)
The price at the end of a day; our bot trades on it.

Consumer group
Our team's listening identity on Kafka (`group-mn2631`).

Dashboard
Our local website showing the bot's state at 127.0.0.1:8080.

DP charge
₹15.34 charged on every sell.

Drawdown
How far wealth has fallen from its highest point so far.

Equity curve
A chart of total wealth over time.

FIFO
First in, first out: the oldest shares are counted as sold first.

Gateway (REST)
The web service where orders are placed and the portfolio is read.

Heartbeat / pre-flight
The roll call before the run; the bot must answer each number within 15 s.

Journal
The bot's diary of every decision and why it was made.

Kafka
The messaging system that streams prices and news to us.

Latency
Time from prices arriving to our order reaching the exchange.

LTCG / STCG
Long-term (over 365 days, 10%) and short-term (15%) capital-gains tax.

Overfitting
Tuning a rule so closely to past data that it fails on new data.

Placebo test
Re-running a test with the signal scrambled to check a result is not luck.

Portfolio
All our cash and shares.

Position
The shares we hold in one company.

Rate limit
At most 120 requests a minute (we use 100).

Realised / unrealised
Profit locked in by selling / profit still on paper.

Risk gate
The 12-point checklist every order must pass.

Sandbox / live run
Practice market (loops old data) / the real scored run.

Sharpe ratio
Return divided by how bumpy the ride was; higher is better.

SMA8
Simple moving average: the average of the last 8 closing prices.

Tax-loss harvesting
Selling a losing stock before 31 March to reduce the year's taxable profit.

Tick
One price message for one stock for one day.

Ticker

A company's short code, e.g. HAVN.

UNCERTAIN

An order with no clear answer; the stock is frozen until the portfolio is re-read.

Win rate

The share of sales that made a profit.

Token

The team's secret key for the gateway. Never share it.

Veto (news)

Skipping a buy because recent headlines scored −2 or worse.

IF YOU REMEMBER ONLY FIVE THINGS

1. The bot buys when a price closes clearly above its 8-day average and sells when it closes clearly below.
2. It skips a buy if the company's news over the last 3 days was bad, and never delays a sell.
3. Every order passes a 12-point safety checklist, and the exchange's records always win.
4. Every fee and the tax are counted; every decision is explained in the journal.
5. All of it was tested on 500 recorded days with the real code: +34.4%, worst fall 10%, and we report what failed.